

# **DSA0501 – Comprehensive Data Exploration, Analysis and Visualization Using Pandas and Matplotlib**

## **1. Introduction**

Data analysis helps convert raw data into useful information for decision-making. In this assignment, Python libraries such as Pandas and Matplotlib are used to import, explore, clean, analyze, and visualize datasets. The analysis focuses on identifying patterns, relationships, correlations, distributions, outliers, and time-based trends.

Pandas is useful for handling tabular data, while Matplotlib provides different visualization techniques such as line charts, bar charts, histograms, scatter plots, and box plots. Advanced techniques such as scatter matrices, lag plots, autocorrelation plots, and bootstrap plots can also be used to obtain deeper insights.

## **2. Objectives**

The main objectives are:

1. To import datasets using Pandas.
2. To explore and understand dataset characteristics.
3. To clean and preprocess data.
4. To perform filtering, grouping, and statistical analysis.
5. To combine multiple datasets using joining techniques.
6. To identify correlations and outliers.
7. To study distributions and relationships between variables.
8. To create suitable visualizations using Pandas and Matplotlib.
9. To analyze time-based patterns using advanced plots.
10. To evaluate which visualization techniques are most effective for communicating data insights.

### 3. Data Importing and Exploration

#### Python Code

```
import pandas as pd
import matplotlib.pyplot as plt

data = {
    "Student": ["A", "B", "C", "D", "E"],
    "Attendance": [90, 85, 75, 95, 65],
    "Marks": [85, 78, 60, 92, 55],
    "Assignment": [90, 80, 65, 95, 50]
}

df = pd.DataFrame(data)
print(df)
print(df.head())
print(df.info())
print(df.describe())
print(df.isnull().sum())
```

#### Output:

```
...   Student  Attendance  Marks  Assignment
0        A           90      85           90
1        B           85      78           80
2        C           75      60           65
3        D           95      92           95
4        E           65      55           50
   Student  Attendance  Marks  Assignment
0        A           90      85           90
1        B           85      78           80
2        C           75      60           65
3        D           95      92           95
4        E           65      55           50
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Student         5 non-null      object
1   Attendance       5 non-null      int64
2   Marks            5 non-null      int64
3   Assignment       5 non-null      int64
dtypes: int64(3), object(1)
memory usage: 292.0+ bytes
```

| None       | Attendance | Marks     | Assignment |
|------------|------------|-----------|------------|
| count      | 5.000000   | 5.000000  | 5.000000   |
| mean       | 82.000000  | 74.000000 | 76.000000  |
| std        | 12.041595  | 15.953056 | 18.506756  |
| min        | 65.000000  | 55.000000 | 50.000000  |
| 25%        | 75.000000  | 60.000000 | 65.000000  |
| 50%        | 85.000000  | 78.000000 | 80.000000  |
| 75%        | 90.000000  | 85.000000 | 90.000000  |
| max        | 95.000000  | 92.000000 | 95.000000  |
| Student    | 0          |           |            |
| Attendance | 0          |           |            |
| Marks      | 0          |           |            |
| Assignment | 0          |           |            |
| dtype:     | int64      |           |            |

## Explanation

`head()` displays the first records, `info()` provides information about columns and data types, and `describe()` provides statistical values such as mean, minimum, maximum, and standard deviation.

## 4. Data Processing, Filtering and Grouping

### Python Code:

```
high_marks = df[df["Marks"] > 75]
print("Students with marks above 75:")
print(high_marks)
print("Average Marks:", df["Marks"].mean())
print("Average Attendance:", df["Attendance"].mean())
df["Performance"] = [
    "Good", "Good", "Average", "Excellent", "Average"
]
grouped = df.groupby("Performance")["Marks"].mean()
print("\nAverage Marks by Performance:")
print(grouped)
```

### Output:

```

... Students with marks above 75:
      Student  Attendance  Marks  Assignment
0          A           90     85          90
1          B           85     78          80
3          D           95     92          95
Average Marks: 74.0
Average Attendance: 82.0

Average Marks by Performance:
Performance
Average      57.5
Excellent    92.0
Good         81.5
Name: Marks, dtype: float64

```

## Explanation

Filtering helps focus on specific records, while grouping helps compare categories. These operations are important for extracting meaningful information from datasets. The assignment specifically expects grouping, filtering, joining, and focusing data appropriately.

## 5. Joining Multiple Datasets

### Python Code:

```

student = pd.DataFrame({
    "StudentID": [1, 2, 3],
    "Name": ["Anu", "Ravi", "Priya"]
})
marks = pd.DataFrame({
    "StudentID": [1, 2, 3],
    "Marks": [85, 78, 92]
})
result = pd.merge(
    student,
    marks,
    on="StudentID"

```

```
)  
print(result)
```

**Output:**

| ... | StudentID | Name  | Marks |
|-----|-----------|-------|-------|
| 0   | 1         | Anu   | 85    |
| 1   | 2         | Ravi  | 78    |
| 2   | 3         | Priya | 92    |

**Explanation**

`pd.merge()` combines two datasets using a common column. Joining is useful when information about the same entities is stored in different tables.

## 6. Correlation and Outlier Analysis

**Correlation**

**Python Code:**

```
correlation = df[  
    ["Attendance", "Marks", "Assignment"]  
].corr()  
print(correlation)
```

**Output:**

| ...        | Attendance | Marks    | Assignment |
|------------|------------|----------|------------|
| Attendance | 1.000000   | 0.982560 | 0.998426   |
| Marks      | 0.982560   | 1.000000 | 0.982254   |
| Assignment | 0.998426   | 0.982254 | 1.000000   |

**Interpretation**

Correlation values range from -1 to +1.

- Near +1 → strong positive relationship

- Near -1 → strong negative relationship
- Near 0 → weak relationship

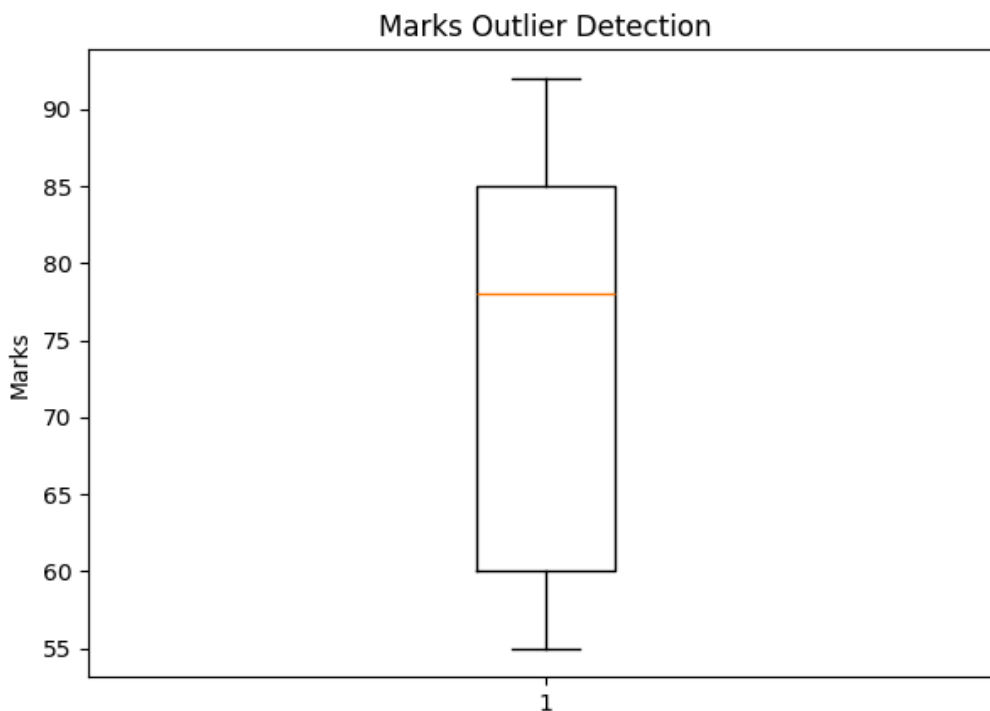
For example, if attendance and marks have a positive correlation, students with higher attendance may generally have higher marks.

## Outlier Detection

### Python Code:

```
plt.figure(figsize=(7, 5))  
plt.boxplot(df["Marks"])  
plt.title("Marks Outlier Detection")  
plt.ylabel("Marks")  
plt.show()
```

### Output:



### Explanation

A box plot helps identify values that are unusually high or low compared with the rest of the data. The assignment requires identification and interpretation of correlations, distributions, and outliers.

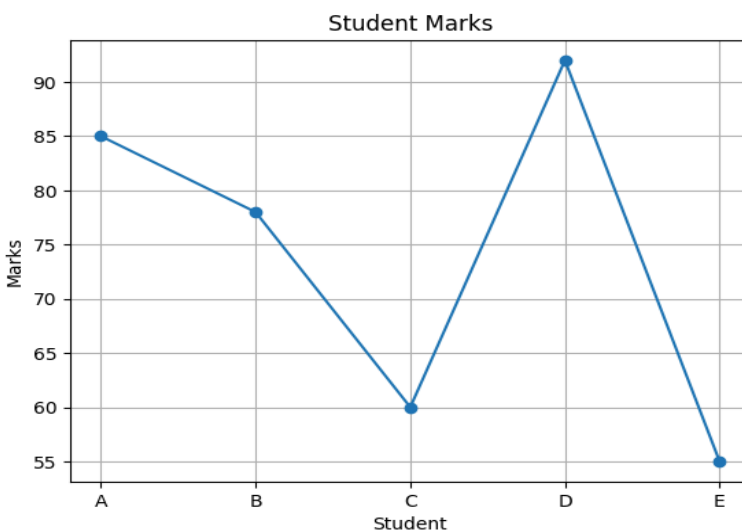
## 7. Data Visualization

### Line Chart

#### Python Code:

```
plt.plot(  
    df["Student"],  
    df["Marks"],  
    marker="o"  
)  
plt.title("Student Marks")  
plt.xlabel("Student")  
plt.ylabel("Marks")  
plt.grid()  
plt.show()
```

#### Output:

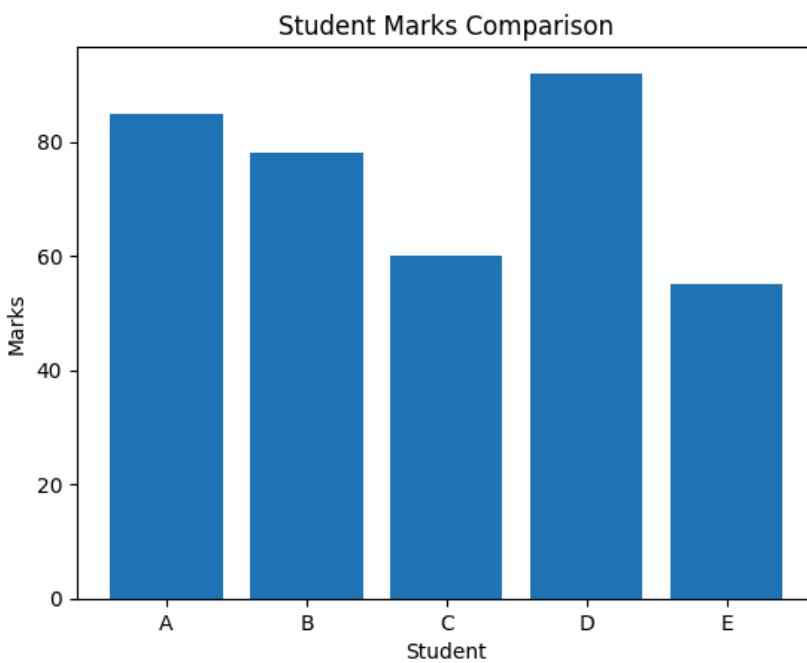


### Bar Chart

```
plt.bar(
```

```
df["Student"],  
df["Marks"]  
)  
plt.title("Student Marks Comparison")  
plt.xlabel("Student")  
plt.ylabel("Marks")  
plt.show()
```

**Output:**



## Histogram

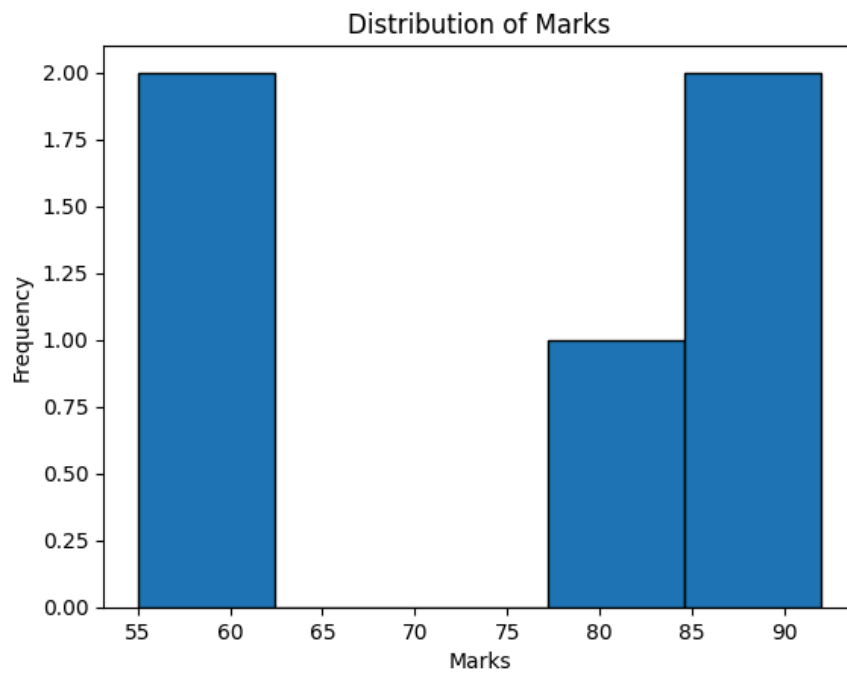
**Python Code:**

```
plt.hist(  
    df["Marks"],  
    bins=5,  
    edgecolor="black"  
)  
plt.title("Distribution of Marks")  
plt.xlabel("Marks")  
plt.ylabel("Frequency")
```



```
plt.show()
```

**Output:**

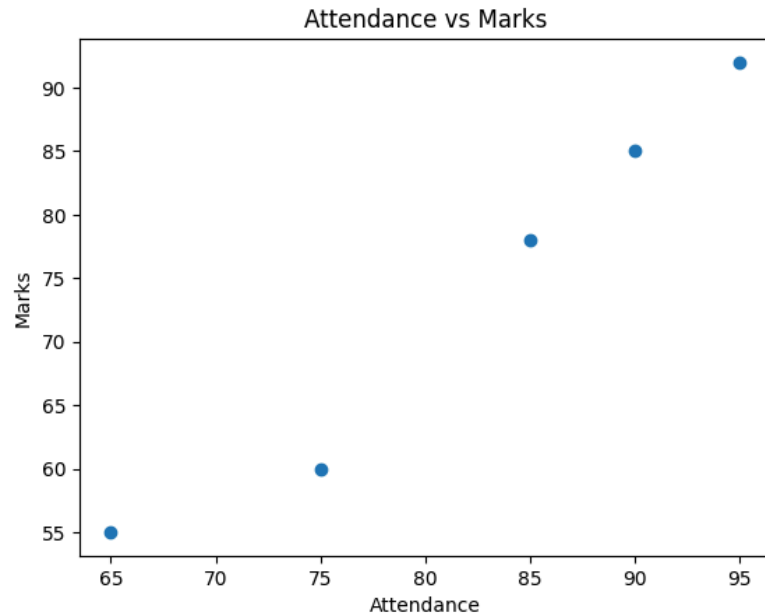


**Scatter Plot**

**Python Code:**

```
plt.scatter(  
    df["Attendance"],  
    df["Marks"]  
)  
plt.title("Attendance vs Marks")  
plt.xlabel("Attendance")  
plt.ylabel("Marks")  
plt.show()
```

**Output:**



### Visualization Interpretation

- Line chart → useful for trends.
- Bar chart → useful for category comparisons.
- Histogram → useful for distributions.
- Scatter plot → useful for relationships.
- Box plot → useful for detecting outliers.

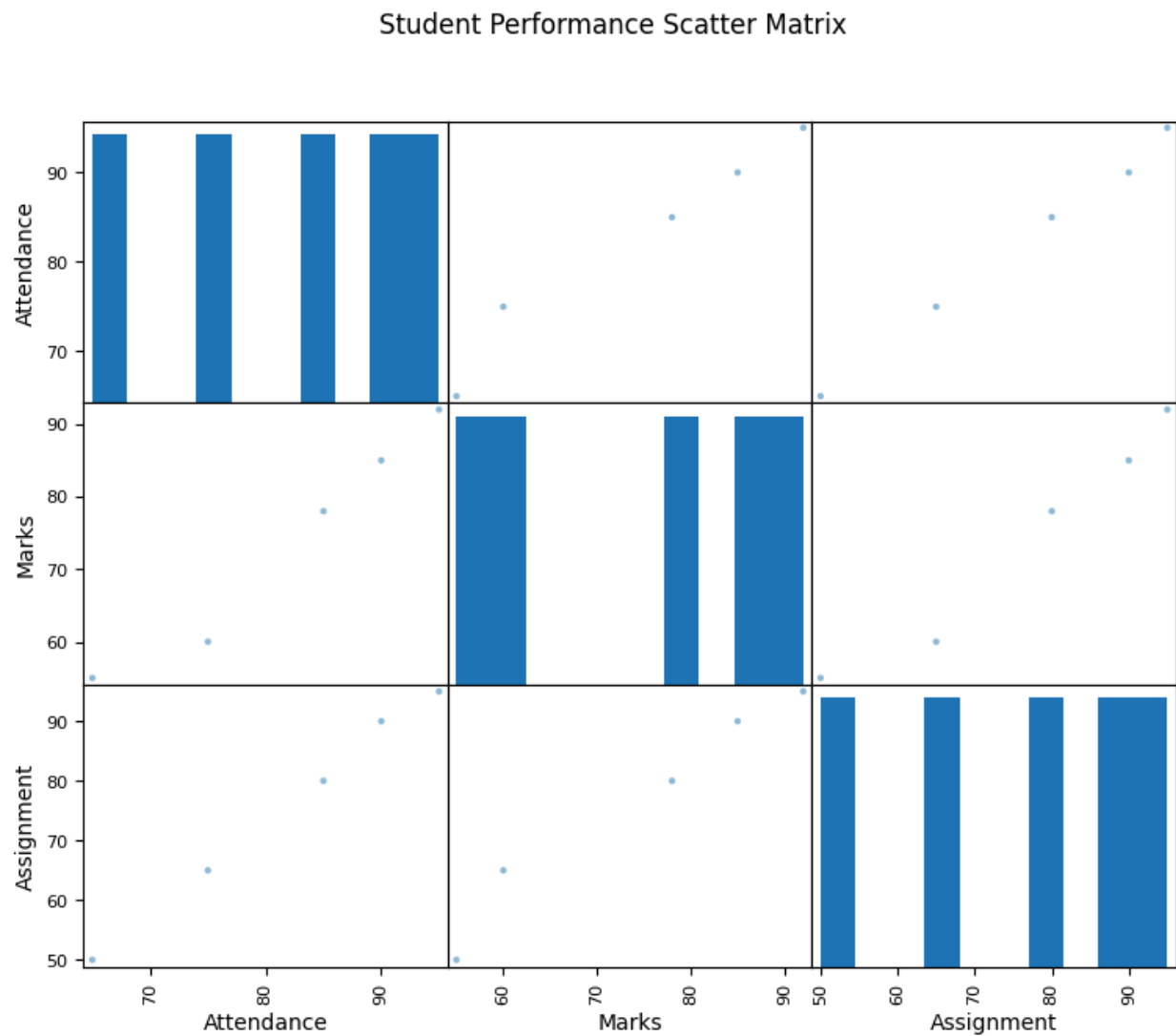
### 8. Scatter Matrix

#### Python Code:

```
pd.plotting.scatter_matrix(  
    df[  
        ["Attendance",  
         "Marks",  
         "Assignment"]  
    ],  
    figsize=(9, 7)
```

```
)  
plt.suptitle("Student Performance Scatter Matrix")  
plt.show()
```

**Output:**



## Interpretation

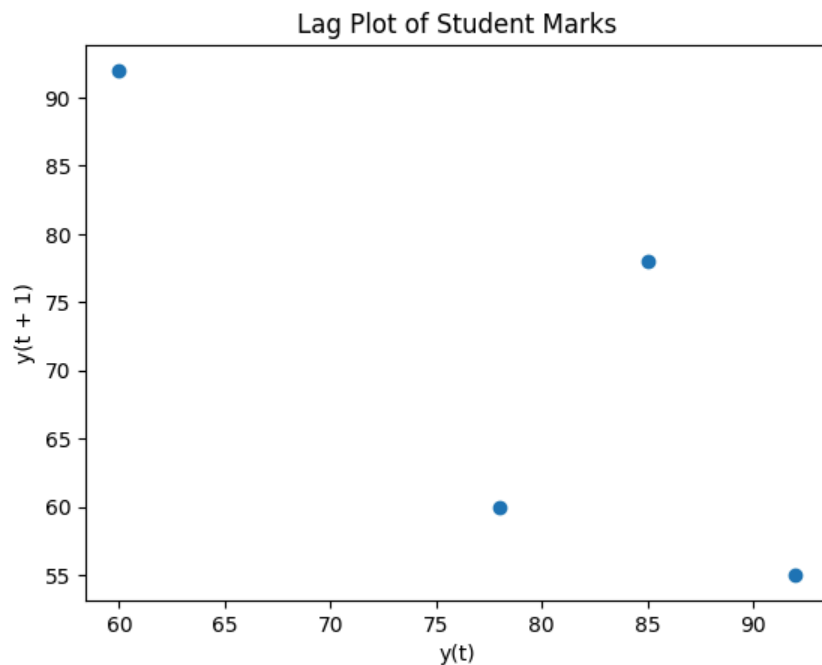
A scatter matrix displays relationships between several numerical variables at the same time. It helps identify which variables have strong relationships and whether any unusual observations are present.

## 9. Lag Plot

### Python Code:

```
from pandas.plotting import lag_plot
lag_plot(df["Marks"])
plt.title("Lag Plot of Student Marks")
plt.show()
```

### Output:



### Interpretation

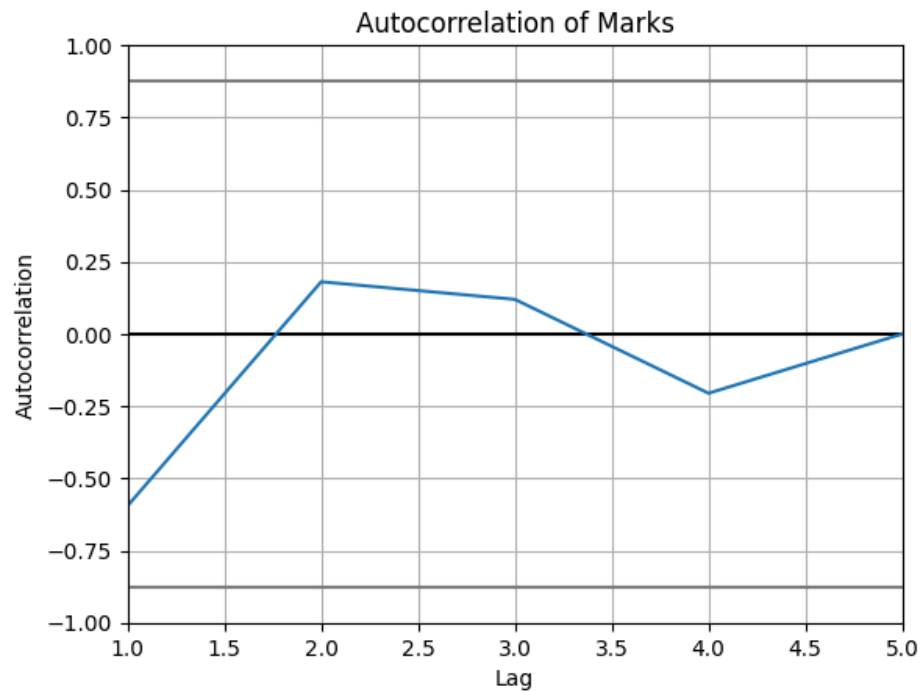
A lag plot compares the current observation with a previous observation. If the points show a clear pattern, the current values may be related to previous values.

## 10. Autocorrelation Plot

### Python Code:

```
from pandas.plotting import autocorrelation_plot
autocorrelation_plot(df["Marks"])
plt.title("Autocorrelation of Marks")
plt.show()
```

**Output:**



## Interpretation

An autocorrelation plot helps identify whether observations in a sequence are related to previous observations. Strong positive autocorrelation means similar values tend to occur close together.

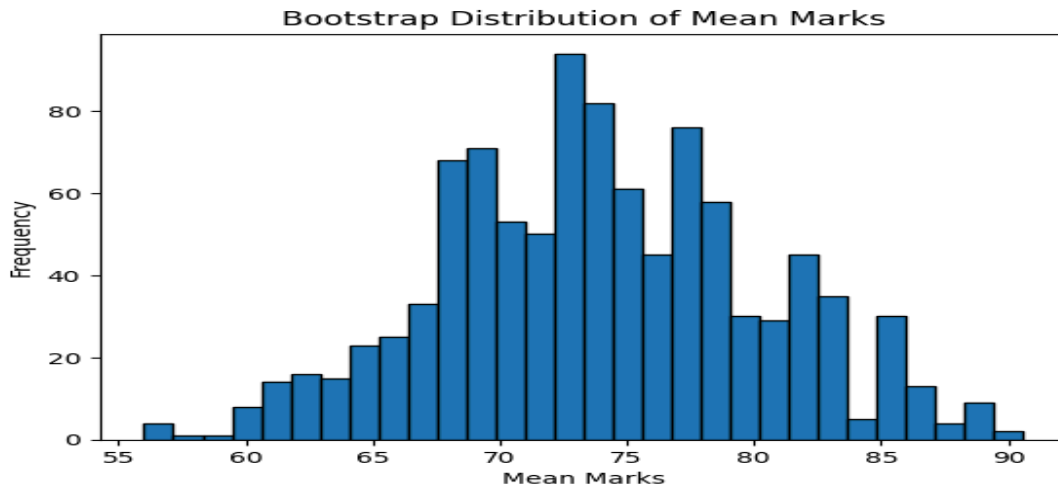
The assignment specifically includes scatter matrices, lag plots, and autocorrelation plots as advanced visualization techniques.

## 11. Bootstrap Analysis

**Python Code:**

```
import numpy as np
import matplotlib.pyplot as plt
marks = df["Marks"].values
bootstrap_means = []
for i in range(1000):
    sample = np.random.choice(
        marks,
        size=len(marks),
        replace=True
    )
    bootstrap_means.append(
        np.mean(sample)
    )
plt.hist(
    bootstrap_means,
    bins=30,
    edgecolor="black"
)
plt.title("Bootstrap Distribution of Mean Marks")
plt.xlabel("Mean Marks")
plt.ylabel("Frequency")
plt.show()
```

**Output:**



## Interpretation

Bootstrap sampling repeatedly creates samples from the original data and calculates the mean. The resulting distribution shows the variability and uncertainty of the estimated mean.

## 12. Overall Findings

The analysis demonstrates that different visualization techniques are suitable for different types of information.

- Line plots clearly show changes and trends.
- Bar charts are effective for comparing categories.
- Histograms show the distribution of numerical data.
- Scatter plots show relationships between two variables.
- Box plots help identify outliers.
- Scatter matrices provide a multivariable view.
- Lag and autocorrelation plots help identify time-dependent patterns.
- Bootstrap plots help understand uncertainty.

## 13. Conclusion

This assignment demonstrates how Pandas and Matplotlib can be used for complete data exploration and visualization. The process starts with importing and understanding the dataset, followed by cleaning, filtering, grouping, joining, statistical analysis, correlation analysis, and outlier detection.

Different visualization techniques provide different types of insights. Basic charts are easier to understand, while advanced techniques such as scatter matrices, lag plots, autocorrelation plots, and bootstrap plots provide deeper analytical information. Therefore, selecting the correct visualization according to the data is important for effective communication and reliable decision-making.

### **Pseudocode**

START

1. Import Pandas, NumPy and Matplotlib libraries.
2. Create or load the dataset.
3. Read the dataset using Pandas.
4. Display the first few rows of the dataset.
5. Check:
  - Number of rows and columns
  - Column names
  - Data types
  - Missing values
  - Duplicate values
6. Clean the dataset:



- Handle missing values
- Remove duplicate records
- Correct incorrect data types

7. Perform data processing:

- Filter required records
- Group data based on categories
- Calculate mean, maximum and minimum values

8. If multiple datasets are available:

- Identify the common column
- Join/merge the datasets

9. Perform data analysis:

- Calculate correlations
- Identify outliers
- Study data distributions
- Analyze relationships between variables

10. Create visualizations:

- Draw line chart
- Draw bar chart
- Draw histogram
- Draw scatter plot

- Draw box plot

11. Perform advanced visualization:

- Create scatter matrix
- Create lag plot
- Create autocorrelation plot
- Create bootstrap distribution

12. Interpret the graphs:

- Identify trends
- Identify relationships
- Identify outliers
- Identify patterns

13. Compare the visualization techniques based on:

- Suitability
- Easy interpretation
- Pattern identification
- Outlier detection

14. Write the important findings and conclusions.

15. Suggest suitable visualization techniques for real-world data analysis and decision-making.

END

## **14. Student Reflection**

### **What I Learned**

I learned how to import datasets using Pandas and perform data exploration, cleaning, filtering, grouping, and statistical analysis. I also learned how visualization can make complex data easier to understand.

### **Techniques Learned**

The most useful techniques were Pandas DataFrame operations, bar charts, line plots, scatter plots, histograms, and correlation analysis. Advanced plots such as scatter matrices and autocorrelation plots helped me understand relationships and patterns in greater detail.

### **Challenges Faced**

I faced difficulties while handling data types, missing values, plotting errors, and understanding the correct visualization for different datasets. These challenges helped me improve my Python programming and debugging skills.

### **Data Insights**

Correlation helped me understand relationships between variables, while box plots helped identify unusual values. Charts and graphs made trends, distributions, and comparisons easier to identify.

### **Real-World Application**

These techniques can be applied to real-world areas such as healthcare, education, finance, sales, transportation, and business analytics to support data-driven decisions.

### **Further Learning**

If given more time, I would explore interactive dashboards, larger real-world datasets, advanced statistical methods, and machine-learning techniques to improve the analysis.

